

CineMax

Sistema di gestione di un piccolo cinema monosala (200 posti)

Manuale Tecnico

Laboratorio Interdisciplinare A — a.a. 2025/2026

Università degli Studi dell'Insubria — Corso di Studio Triennale in Informatica

Autori: D'Onofrio Greta (matr. 766016, sede CO) — Tricarico Nicolò (matr. 767095, sede CO)

Versione del documento: 1.0

Indice

1. Introduzione e struttura dell'applicazione
2. Scelte architettureali
3. Strutture dati utilizzate
4. Scelte algoritmiche
5. Formato dei file e loro gestione
6. Compilazione, esecuzione e generazione della Javadoc
7. Interpretazioni della specifica e limiti
8. Sitografia / Bibliografia

1. Introduzione e struttura dell'applicazione

CineMax è sviluppato interamente in Java (versione minima 17, testato con JDK 21) e usa esclusivamente la libreria standard: non sono richieste librerie esterne (cartella lib vuota). Tutto il codice è contenuto nel package **cinemax**, come richiesto dalla specifica; il metodo **main** si trova nella classe **cinemax.CineMax**. Il codice è commentato in formato Javadoc per ogni package, classe, attributo e metodo.

Classe	Responsabilità
Film	Dati anagrafici di un film (titolo, genere, regista, anno, durata, età minima)
Proiezione	Un film in una data/ora con un prezzo; calcolo fine e sovrapposizioni
Utente / Ruolo	Utente registrato con ruolo (cliente, proiezionista, bigliettaio)
Prenotazione	Codice univoco, cliente, proiezione, n. posti, costo unitario/totale
GestoreProiezioni	Palinsesto: carica/salva proiezioni.csv, cercaProiezione(), CRUD
GestoreUtenti	Utenti: carica/salva utenti.csv, login, registraCliente()
GestorePrenotazioni	Prenotazioni: carica/salva prenotazioni.csv, codici, posti liberi
CineMax	main() e interfaccia a terminale (menù per ruolo, validazione input)
CsvUtil, Sicurezza	Parsing/scrittura CSV con virgolette; hash SHA-256 delle password

2. Scelte architetturali

L'applicazione adotta una separazione a livelli ispirata al pattern **MVC**, nella misura in cui le competenze del primo anno lo consentono: le classi del *modello* (Film, Proiezione, Utente, Prenotazione) rappresentano i dati e non conoscono né i file né l'interfaccia; le classi *gestore* incapsulano la logica applicativa e la persistenza (pattern informalmente noto come *repository*); la classe CineMax realizza la sola *vista/controllo* a terminale. Questa separazione rende il codice testabile e predispone naturalmente il progetto alla futura evoluzione del Lab B (sostituzione dei CSV con un database senza toccare modello e interfaccia). Le classi di utilità CsvUtil e Sicurezza hanno costruttore privato e soli metodi statici.

Un'invariante centrale dell'architettura: essendo il cinema **monosala** e vietate le sovrapposizioni, la **data e ora di inizio identifica univocamente una proiezione**. Le prenotazioni riferenziano quindi la proiezione tramite la sua data/ora, evitando l'introduzione di identificatori artificiali e mantenendo i file leggibili.

3. Strutture dati utilizzate

- **ArrayList<Proiezione>** mantenuta ordinata per data/ora (Comparator su LocalDateTime): l'ordinamento rende immediata la presentazione cronologica degli elenchi e naturale la verifica delle sovrapposizioni.
- **LinkedHashMap<String, Utente>** indicizzata per username: il login e il controllo di unicità dello username costano $O(1)$ in media, e l'ordine di inserimento preserva l'ordine del file.
- **ArrayList<Prenotazione>** per le prenotazioni, con scansioni lineari per i filtri di ricerca. Le date sono rappresentate con **java.time** (LocalDate, LocalDateTime), che offre confronti e parsing robusti ed evita gli errori tipici delle date come stringhe.

4. Scelte algoritmiche

Ricerca con criteri combinabili. `cercaProiezione()` e `cercaPrenotazione()` ricevono un parametro per criterio, dove *null* significa "criterio non usato": un'unica scansione $O(n)$ applica in AND solo i filtri attivi (titolo parziale case-insensitive con `contains`, uguaglianza sul genere, intervalli chiusi su date e prezzi). Con $n \approx 10^3$ proiezioni la ricerca è istantanea e non giustifica indici più complessi.

Sovrapposizione di proiezioni. Due intervalli $[inizio, fine)$ si sovrappongono se e solo se $inizioA < fineB$ e $inizioB < fineA$; la fine è calcolata come $inizio + durata$ del film. L'inserimento verifica la condizione contro tutte le proiezioni esistenti, $O(n)$.

Posti liberi. `postiLiberi(dataOra) = 200 - somma dei posti delle prenotazioni per quella proiezione` (scansione $O(m)$); la creazione di una prenotazione è accettata solo se i posti richiesti non superano i liberi. **Codici univoci:** generazione casuale nel formato PRN-XXXXXXX su un alfabeto di 32 caratteri non ambigui ($32^7 \approx 10^{12}$ combinazioni) con verifica di collisione e rigenerazione, quindi unicità garantita per costruzione.

Password. Alla registrazione la password è trasformata con SHA-256 (`java.security.MessageDigest`) e sul file viaggia solo l'hash esadecimale; il login confronta gli hash. La password in chiaro non è mai memorizzata.

5. Formato dei file e loro gestione

I tre archivi risiedono nella cartella **data** (percorso relativo alla cartella di lavoro; un percorso alternativo può essere passato come primo argomento al programma). Si è scelto il **formato testuale CSV** anziché la serializzazione di oggetti: è il formato del file draft fornito dal docente, è ispezionabile e modificabile con qualsiasi editor o foglio di calcolo ed è indipendente dalla versione delle classi Java. Il parser di `CsvUtil` gestisce i campi tra doppi apici contenenti virgole (es. "Monsters, Inc.") e gli apici raddoppiati.

```
data/proiezioni.csv  data_ora_proiezione,titolo_film,genere,regista,anno,
                     durata_minuti,eta_minima,prezzo_biglietto
data/utenti.csv      nome,cognome,username,password_hash,data_nascita,
                     domicilio,ruolo      (data_nascita facoltativa)
data/prenotazioni.csv codice,username_cliente,data_ora_proiezione,
                     numero_posti,costo_unitario
```

I file sono caricati integralmente in memoria all'avvio e riscritti per intero dopo ogni operazione di modifica (write-through): la strategia è semplice, mantiene i file sempre coerenti con lo stato in memoria ed è adeguata alle dimensioni in gioco. Le righe malformate vengono ignorate con un avviso, senza impedire l'avvio; `prenotazioni.csv` viene creato automaticamente se assente. I costi sono scritti con `Locale.US` per garantire il punto decimale. Il numero di posti prenotati è volutamente *non* duplicato nel file delle proiezioni: i posti liberi sono sempre derivati dalle prenotazioni, eliminando ogni possibile incoerenza tra archivi.

6. Compilazione, esecuzione e generazione della Javadoc

```
javac -d out src/cinemax/*.java
jar --create --file bin/CineMax.jar --main-class cinemax.CineMax -C out .
java -jar bin/CineMax.jar
javadoc -d doc/javadoc -private -encoding UTF-8 -charset UTF-8 src/cinemax/*.java
```

I comandi vanno eseguiti dalla cartella radice del progetto. La Javadoc generata in `doc/javadoc` è parte integrante di questo manuale tecnico e documenta ogni package, classe, interfaccia, metodo e attributo, con i tag `@param`, `@return`, `@throws`, `@author` e `@see`.

7. Interpretazioni della specifica e limiti

Due punti della specifica sono stati interpretati e la scelta è qui motivata. (1) La cancellazione di una prenotazione è testualmente ammessa "a patto che la data di proiezione sia precedente la data odierna"; si è ritenuto un refuso (annullerebbe solo prenotazioni di spettacoli già avvenuti) e si è implementato il vincolo coerente con la modifica: la cancellazione è ammessa solo per proiezioni **successive** alla data odierna. (2) L'inserimento richiede posti "minori" dei disponibili: si è implementato il confronto $\leq$ (è possibile prenotare esattamente gli ultimi posti liberi), interpretazione naturale dell'intento. Nella modifica di una prenotazione si richiede inoltre che la nuova proiezione riguardi lo stesso film ("cambio data") e abbia posti sufficienti. Limiti generali: nessuna gestione della concorrenza né architettura client/server (escluse dalla specifica), interfaccia solo testuale, hash delle password senza salt.

8. Sitografia / Bibliografia

- Oracle, "Java SE API Specification" (java.time, java.nio.file, java.security). Online: <https://docs.oracle.com/en/java/javase/21/docs/api/>
- Oracle, "Javadoc Tool". Online: <https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html>
- RFC 4180, "Common Format and MIME Type for CSV Files". Online: <https://www.rfc-editor.org/rfc/rfc4180>
- G. Meroni, materiale del corso "Laboratorio Interdisciplinare A", Università degli Studi dell'Insubria, a.a. 2025/2026.